# Lab3: Score Distillation Sampling Implementation

*Your Name (Student ID)*

October 01, 2026

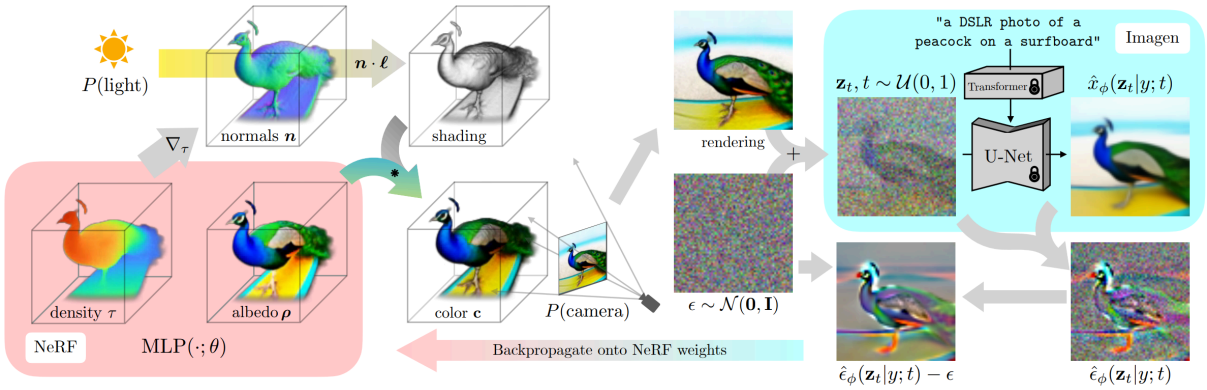



Figure 1: **Overview of score distillation.** Describe it briefly; cite its source if not yours.

## Introduction

What problem does score distillation address, and how is it different from ordinary diffusion sampling? Keep it to a few sentences; you may refer to Figure 1.

State the goal of this assignment: which methods you implemented, what you generated, and how you evaluated the results.

(Optional) Briefly outline the rest of the report.

## Methodology

Introduce the background and notation shared by the three methods (e.g. the pretrained diffusion model [6], the forward noising process [4], classifier-free guidance [9]), so the following subsections can refer to them.

### 1. Score Distillation Sampling (SDS)

**Concept.** In your own words: what is being optimized in SDS [1], where does the gradient come from, and why is it a reasonable objective? Point out the approximation that makes SDS practical.

**Implementation.** Paste your `get_sds_loss()` into the code block below, then walk through it: the main steps, the design choices you made and why, and anything that was tricky to get right.

```
def get_sds_loss(
    self, latents, text_embeddings,
    guidance_scale=7.5,
```

```
):
    # Paste your implementation here
    ...
```

### 2. Variational Score Distillation (VSD)

**Concept.** Which weakness of SDS does VSD [2] try to fix? What extra component does it introduce (cf. LoRA [7]), and what is that component trained to do?

**Implementation.** Paste your `get_vsd_loss()` into the code block below, then walk through it: how the extra component is set up and updated, how its loss and the distillation loss interact, and which parts receive gradients.

```
def get_vsd_loss(
    self, latents, text_embeddings,
    guidance_scale=7.5, lora_loss_weight=1.0,
):
    # Paste your implementation here
    ...
```

### 3. Score Distillation via Inversion (SDI)

**Concept.** Which weakness of SDS does SDI [3] try to fix? How does it obtain the noisy sample differently (cf. DDIM [5]), and how does its timestep schedule differ from SDS?

**Implementation.** Paste your `invert_noise()` (DDIM inversion) and `get_sdi_loss()` into the two code blocks below, then walk through them: how your inversion loop works, how the timestep changes over the iterations, and how and when the target is updated.

```
@torch.no_grad()
def invert_noise(
    self, latents, target_t, text_embeddings,
    guidance_scale=-7.5, n_steps=10, eta=0.3,
):
    # Paste your DDIM inversion here
    ...
```

```
def get_sdi_loss(
    self, latents, text_embeddings,
    guidance_scale=7.5,
    current_iter=0, total_iters=500,
    inversion_guidance_scale=-7.5,
    inversion_n_steps=10, inversion_eta=0.3,
    update_interval=25,
):
    # Paste your implementation here
    ...
```

# Experiments & Results

## 1. Quantitative Results

Which method has the best average CLIP score (Table 1)? Are there prompts where the ranking changes, and why might that be? Does the score agree with your visual impression?

Table 1: **CLIP score** (↑, ViT-B/32) per prompt.

| Method | Bus | Boat | Cabin | Church | Villa | Castle | Burger | Dog | Cat | Car | Avg. |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **SDS** | – | – | – | – | – | – | – | – | – | – | – |
| **VSD** | – | – | – | – | – | – | – | – | – | – | – |
| **SDI** | – | – | – | – | – | – | – | – | – | – | – |

## 2. Qualitative Results

Compare the methods visually and point to concrete examples in Figure 2. What are the typical strengths and failure cases of each method?




Figure 2: **Qualitative comparison** of SDS, VSD and SDI on the 10 evaluation prompts.

## 3. Ablation Study

Briefly describe the setup: which prompts and values were tested; all other hyperparameters follow Table 3. Table 2 lists the CLIP scores of all three ablations.

Table 2: **Ablation study.** Average CLIP score (↑).

| *(a) SDS:* `guidance_scale` | | | | | |
|---|---|---|---|---|---|
| Value | 7.5 | 15 | 25 | 50 | 100 |
| CLIP | 0.0000 | 0.0000 | 0.0000 | 0.0000 | 0.0000 |
| *(b) SDI:* `inversion_n_steps` | | | | | |
| Value | 1 | 3 | 5 | 10 | 25 |
| CLIP | 0.0000 | 0.0000 | 0.0000 | 0.0000 | 0.0000 |
| *(c) SDI:* `sdi_update_interval` | | | | | |
| Value | 25 | 75 | 200 | 500 | 1000 |
| CLIP | 0.0000 | 0.0000 | 0.0000 | 0.0000 | 0.0000 |

**SDS:** `guidance_scale`

How do the results change as the guidance scale increases (Figure 3, Table 2 (a)), and why? Which value works best, and how does it compare with the guidance scale typically used for ordinary sampling?

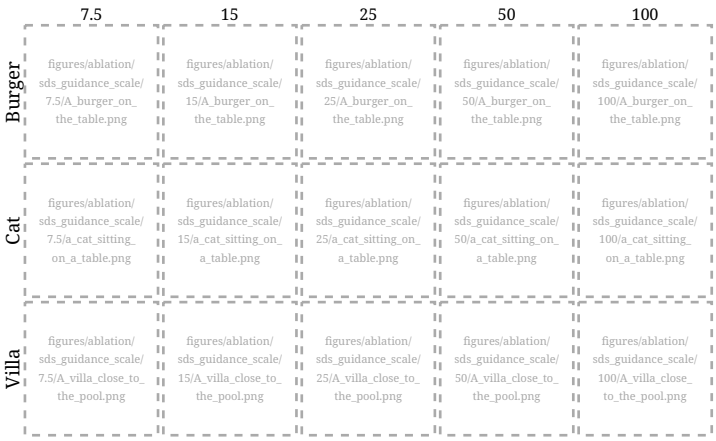



Figure 3: **Ablation on SDS `guidance_scale`.**

**SDI: `inversion_n_steps`**

How do the results and the runtime change with the number of inversion steps (Figure 4, Table 2 (b)), and why? Is there a trade-off?

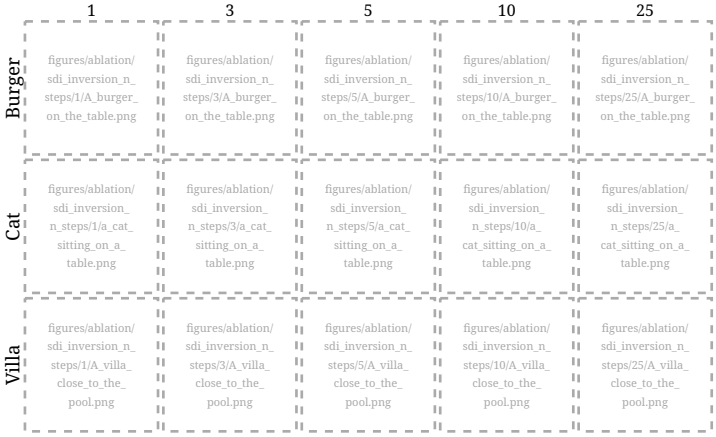



Figure 4: **Ablation on SDI `inversion_n_steps`.**

**SDI: `sdi_update_interval`**

How do the results change with the update interval (Figure 5, Table 2 (c)), and why? What happens at the two extremes?

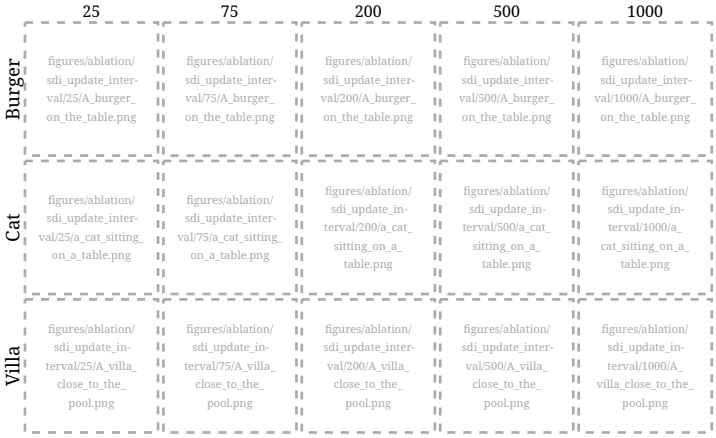



Figure 5: **Ablation on SDI `sdi_update_interval`.**

### 4. Hyperparameters

Fill in Table 3 with the settings you actually used (defaults are pre-filled from `main.py` / README) and justify any value you changed.

Table 3: **Hyperparameters** for each task.

| Hyperparameter | SDS | VSD | SDI |
|---|---|---|---|
| `guidance_scale` | 25 | 7.5 | 7.5 |
| `lr` | 0.01 | 0.01 | 0.005 |
| `steps` | 500 | 500 | 1000 |
| `lora_lr` | – | 1e-4 | – |
| `lora_rank` | – | 4 | – |
| `lora_loss_weight` | – | 1.0 | – |
| `lora_weight_decay` | – | 1e-4 | – |
| `inversion_n_steps` | – | – | 10 |
| `inversion_guidance_scale` | – | – | –7.5 |
| `inversion_eta` | – | – | 0.3 |
| `sdi_update_interval` | – | – | 25 |

## Discussion

### 1. Why the Original Diffusion Loss Fails

Explain how you produced the result in Figure 6. Then explain intuitively (at most one equation) why directly optimizing the diffusion loss does not give realistic images, and connect your explanation to what is visible in the figure.

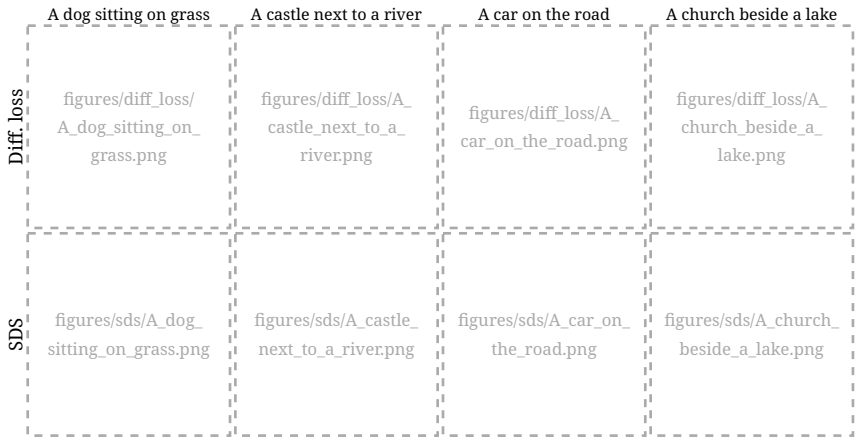



Figure 6: **Original diffusion loss** (top) vs. SDS (bottom).

### 2. Why SDI and VSD Outperform SDS

**SDS.** What are the typical artifacts of SDS in your results, and what in the SDS update causes them?

**VSD.** What does VSD change compared with SDS, and why does that lead to better results? Explain intuitively, without heavy math, and support it with your own figures or numbers.

**SDI.** What does SDI change compared with SDS, and why does that lead to better results? Explain intuitively, without heavy math, and support it with your own figures or numbers.

## Conclusion

Summarize what you implemented, the main quantitative and qualitative findings, the key lessons from the ablations, and your answer to why VSD and SDI outperform SDS.

## Reference

[1] B. Poole, A. Jain, J. T. Barron, and B. Mildenhall. DreamFusion: Text-to-3D using 2D Diffusion. In *Int. Conf. on Learning Representations (ICLR)*, 2023.

[2] Z. Wang, C. Lu, Y. Wang, F. Bao, C. Li, H. Su, and J. Zhu. ProlificDreamer: High-Fidelity and Diverse Text-to-3D Generation with Variational Score Distillation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[3] A. Lukoianov, H. Sáez de Ocáriz Borde, K. Greenewald, V. C. Guizilini, T. Bagautdinov, V. Sitzmann, and J. Solomon. Score Distillation via Reparametrized DDIM. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[4] J. Ho, A. Jain, and P. Abbeel. Denoising Diffusion Probabilistic Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[5] J. Song, C. Meng, and S. Ermon. Denoising Diffusion Implicit Models. In *Int. Conf. on Learning Representations (ICLR)*, 2021.

[6] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-Resolution Image Synthesis with Latent Diffusion Models. In *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[7] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *Int. Conf. on Learning Representations (ICLR)*, 2022.

[8] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning Transferable Visual Models From Natural Language Supervision. In *Int. Conf. on Machine Learning (ICML)*, 2021.

[9] J. Ho and T. Salimans. Classifier-Free Diffusion Guidance. *arXiv preprint arXiv:2207.12598*, 2022.